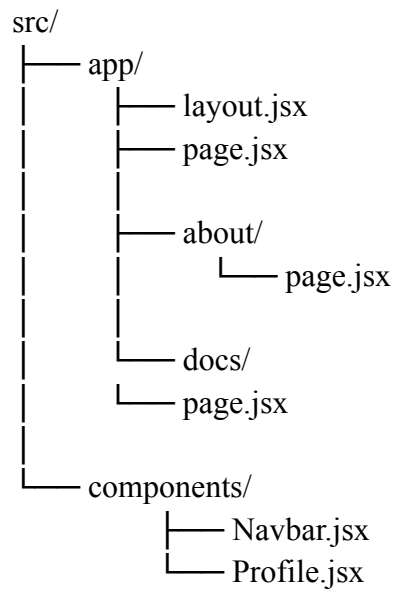


# Next.js App Router — Basic Structure & How It Works

## 1. Basic Project Structure

Next.js App Router ব্যবহার করলে মূল কাজ শুরু হয় `src/app` থেকে।



---

## 2. `page.jsx` কী?

`page.jsx` হলো Next.js-এর **special file**।

এটা সাধারণত কোনো একটি route-এর UI define করে।

`app/page.jsx`  
↓  
/

অর্থাৎ:

`app/page.jsx` → /

আবার:

app/about/page.jsx → /about

এবং:

app/docs/page.jsx → /docs

### Rule

Folder name → URL-এর route segment

page.jsx → সেই route-এর UI

---

## 3. Root **page.jsx** কী?

```
app/  
├── layout.jsx  
└── page.jsx
```

এখানে:

app/page.jsx

হলো **Home route**-এর **page**।

এর URL:

/

এটা অন্য কোনো page-এর parent নয়। বরং **page . jsx** গুলো **parallel route**, একটার child আরেকটা না।

---

## 4. About Page কী?

যদি structure হয়:

```
app/  
├── layout.jsx  
├── page.jsx  
└── about/  
    └── page.jsx
```

তাহলে:

```
app/page.jsx  
  ↓  
  /
```

```
app/about/page.jsx  
  ↓  
  /about
```

খুব গুরুত্বপূর্ণ:

**app/page.jsx** থেকে **about/page.jsx** আসে না।

দুটো আলাদা route-এর page।

Conceptually:

```
      Root Layout  
     /   |   \  
    /    |    \  
  Home  About Docs  
   |     |     |  
  "/"   "/about" "/docs"
```

---

## 5. layout.jsx কী?

layout . jsx-ও Next.js-এর special file।

Root:

app/layout.jsx

হলো **Root Layout**।

এটাকে পুরো application-এর common outer wrapper হিসেবে ভাবতে পারো।

উদাহরণ:

```
export default function RootLayout({ children }) {  
  return (  
    <html lang="en">  
      <body>  
        {children}  
      </body>  
    </html>  
  );  
}
```

---

## 6. children কী?

children হলো একটি prop।

```
export default function RootLayout({ children }) {
```

এখানে:

children

মানে:

বর্তমানে যে route-এর UI render হচ্ছে, সেটা এখানে বসবে।

/ গেলে

Root Layout



children



app/page.jsx

/about গেলে

Root Layout



children



app/about/page.jsx

অর্থাৎ **app/page.jsx** সবসময় Root Layout-এর children নয়।

**Current route** অনুযায়ী **children** পরিবর্তিত হয়।

---

## 7. সবচেয়ে গুরুত্বপূর্ণ ধারণা

ধরো:

```
app/
├── layout.jsx
├── page.jsx
└── about/
    └── page.jsx
```

/ এ গেলে

Root Layout



Home Page

**/about** এ গেলে

Root Layout



About Page

তাই:

Root Layout



Current Route-এর Page

---

## 8. Root Layout-এ Navbar রাখলে কী হয়?

ধরো:

```
components/  
└── Navbar.jsx
```

**Navbar . jsx** হলো **normal React component**।

এটা Next.js-এর special file নয়।

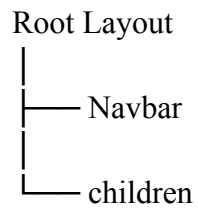
```
export default function Navbar() {  
  return <nav>My Navbar</nav>;  
}
```

তারপর Root Layout-এ:

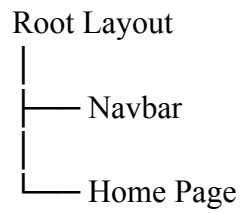
```
import Navbar from "@/components/Navbar";  
  
export default function RootLayout({ children }) {  
  return (  
    <html>  
      <body>  
        <Navbar />  
  
        {children}  
      </body>  
    </html>
```

```
);  
}
```

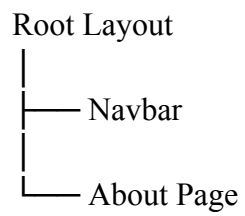
এখন:



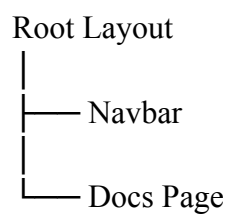
**/** গেলে



**/about** গেলে



**/docs** গেলে



তাই Navbar সব route-এ দেখা যায়।

---

## 9. Page-এর ভিতরে Component

ধরো:

```
components/  
└── Profile.jsx
```

এটা normal React component।

```
export default function Profile() {  
  return <h2>My Profile</h2>;  
}
```

এখন `about/page.jsx` থেকে:

```
import Profile from "@components/Profile";
```

```
export default function About() {  
  return (  
    <div>  
      <h1>About Page</h1>  
  
      <Profile />  
    </div>  
  );  
}
```

তখন hierarchy:

```
Root Layout  
↓  
About Page  
↓  
Profile Component
```

অর্থাৎ:

**Page → Component**

এটাই সাধারণ flow।

---

## 10. Component কি Page-কে call করবে?

সাধারণভাবে:

Page

↓

Component

করা হয়।

কিন্তু:

Component

↓

Page

করা উচিত নয়।

কারণ `page.jsx` হলো Next.js-এর special route entry point।

**Component** থেকে অন্য **page**-এ যেতে চাইলে **page**-কে **import** না করে **navigation** ব্যবহার করো।

যেমন:

```
import Link from "next/link";
```

```
export default function Navbar() {  
  return (  
    <Link href="/about">  
      About  
    </Link>  
  );  
}
```

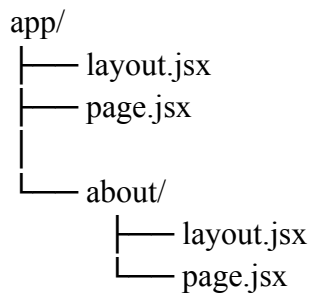
এখানে Navbar `/about` route-এ **navigate** করছে।

About page-কে component হিসেবে import করছে না।

---

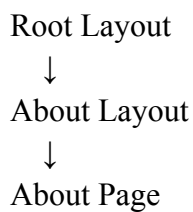
## 11. Nested Layout

এখন ধরো:

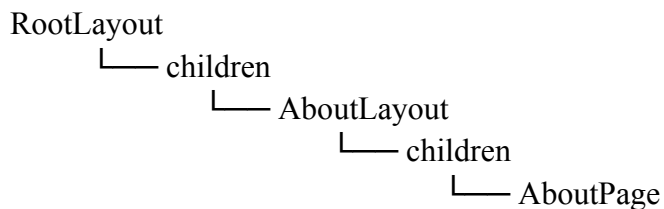


এখানে `about/layout.jsx` হলো **About**-এর নিজস্ব **layout**।

`/about` গেলে hierarchy:

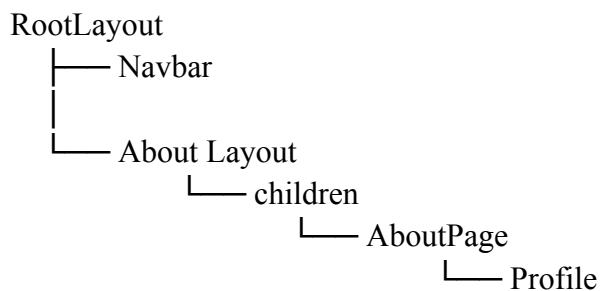


আরও সহজভাবে:



`AboutPage` হলো `AboutLayout`-এর `children`, আর `AboutLayout` হলো `RootLayout`-এর `children`।

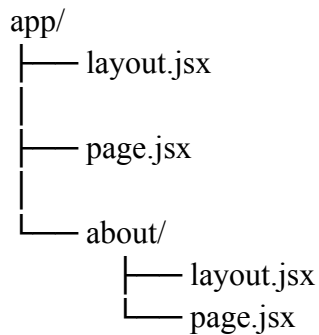
Navbar থাকলে -



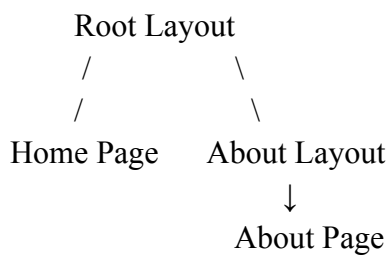
## 12. Root Layout কেন "main"?

Root `layout.jsx` হলো পুরো application-এর root wrapper। কারণ Root Layout application-এর সবচেয়ে বাইরের layout।

ধরো:



তাহলে:

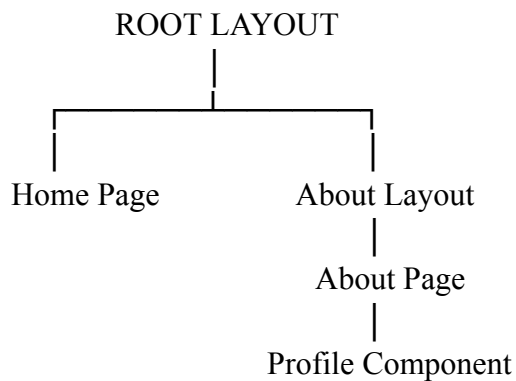


Root Layout সব child route-এর উপরে থাকে।

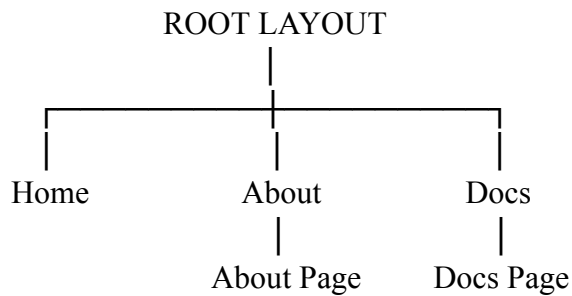
---

## 13. Final Mental Model

Next.js App Router-কে এভাবে মনে রাখো:



আর যদি Docs থাকে:



## 14. সবচেয়ে সহজ Rules

### Rule 1

layout.jsx

→ Next.js-এর **special file**

### Rule 2

page.jsx

→ Next.js-এর **special file**

### Rule 3

Navbar.jsx

Profile.jsx

Card.jsx

→ সাধারণ **React components**

### **Rule 4**

Folder name

→ URL-এর route segment

### **Rule 5**

page.jsx

→ সেই route-এর UI

### **Rule 6**

layout.jsx

→ তার নিচের page/layout-কে wrap করে

### **Rule 7**

**{children}**

→ বর্তমানে যে **route render** হচ্ছে তার **UI**

### **Rule 8**

Page → Component

স্বাভাবিক।

Component → Page

সাধারণত করা হয় না। Link use kora hoy.

## Rule 9

`app/page.jsx` মানে শুধু:

/

এটা `/about`-এর parent নয়।

## Rule 10

`app/about/page.jsx` মানে:

/about

এবং `app/docs/page.jsx` মানে:

/docs

---

## 15. এক লাইনে পুরো Next.js Flow

URL

↓

Next.js route খুঁজে

↓

সেই route-এর page.jsx

↓

প্রয়োজনীয় parent layout-এর ভিতরে বসে

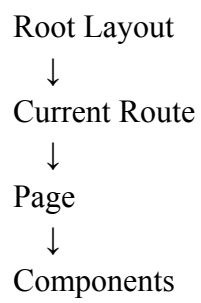
↓

page-এর ভিতরের components render হয়

↓

Browser-এ UI দেখা যায়

সবচেয়ে গুরুত্বপূর্ণ mental picture:



আর nested layout থাকলে:

